

Trees and model averaging

Audrey Poterie

Office B137 building Yves Coppens

audrey.poterie@univ-ubs.fr

2026-2027 - M2 DSMS, M2 SDD

Objectives:

- Be able to identify a supervised learning problem and to propose a statistical approach to answer the question.
- Know and understand tree-based algorithms (specially CART) and ensemble methods such as bagging, random forests and boosting: motivations, principle, main properties.
- Be able both to apply properly CART, bagging, random forests and boosting on a dataset and to understand the results.

Course content

- Chap. 1 : Reminder on supervised statistical learning
- Chap. 2 : Breiman's decision trees
- Chap. 3 : Model averaging: bagging and random forests
- Chap. 4 : Model averaging: boosting (*only for M2 DSMS*)

Organization

- Volume for M2 SDD: 4 CM + 4 TD.
- Volume for M2 DSMS: 5 CM + 5 TD.
- Assessment: short exams + final exam (3h)

Chap. 1 : Reminder on supervised statistical learning

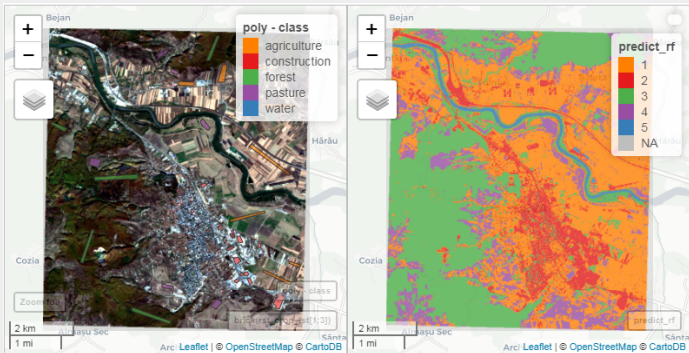
- More and more **complex** (available) data
 - ⇒ Complicated and poorly understood phenomena that we want to explain.
 - ⇒ Various fields: Medicine, Biology, Physics, Finance, etc... **all fields with available data!**
- ↪ Use of **statistical learning approaches**.

What is statistical learning ?

- **To characterize and to predict** phenomena described by many variables based on observed data.
- "*...to extract important patterns and trends and to **understand what the data says***". [Friedman et al., 2008]

Example: Land Cover Classification from Satellite Imagery

Use of a pixel based supervised classification of Sentinel-2 multispectral images

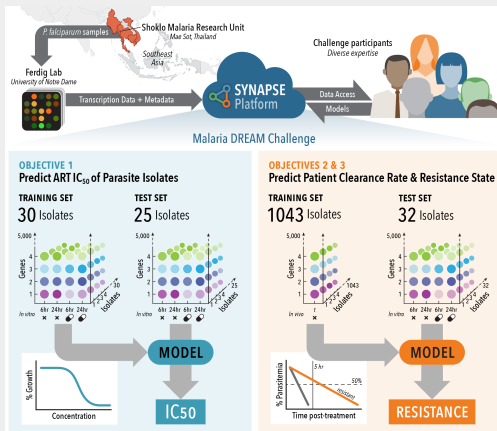


Original image (left) and the model prediction (right).

Source: https://valentinitnelav.github.io/satellite-image-classification-r/#compare_models

Example: Malaria DREAM challenge (summer 2019)

Identify new biomarker to improve the prediction of Artemisinin drug resistance level.

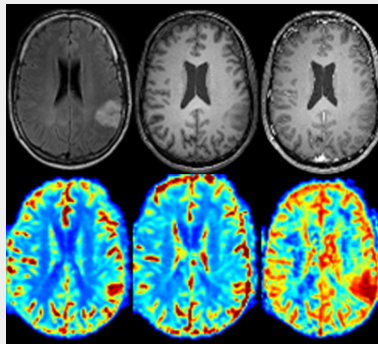
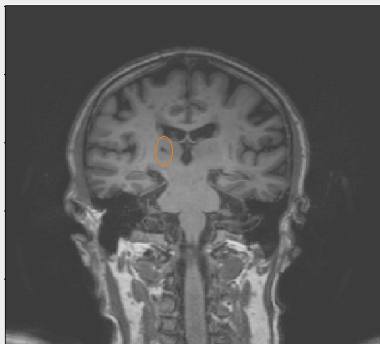


The challenge: <https://www.synapse.org/#!Synapse:syn16924919/wiki/583955>

Melbourne team work: <https://www.synapse.org/#!Synapse:syn20504495/wiki/595348>

Example: Automated detection of brain lacunes

How to identify automatically brain lacunes based on MRI data?



Neuroimaging Team in Centre for Healthy Brain Ageing: <https://cheba.unsw.edu.au/research-groups/neuroimaging>

Example: Iris flower classification

iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

Goal: predict the flower class based on its petal and sepal characteristics (length and width).

Available data : three species (setosa, versicolor et virginica), 50 observations for each kind of species, 4 measured variables (sepal length, petal length, sepal width and petal width).

Mathematical context and terminology

- **Goal:** explain/predict an **output** $y \in \mathcal{Y}$ from **inputs** $\mathbf{x} \in \mathcal{X}$.
- **Available data:** $d_n = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ where the $y_i \in \mathcal{Y}$ denote the outputs and the $\mathbf{x}_i \in \mathcal{X}$ denote the inputs.

Terminology :

- ▷ y : the response, the output, the target.
- ▷ $\mathbf{x} = (x_1, \dots, x_p)$: the inputs, a vector of p predictors/features.
 - Use of bold print to emphasize that $\mathbf{x}$ is a vector.
- ▷ $x_1, \dots, x_j, \dots, x_p$: the components of vector $\mathbf{x}$, the p predictors.
 - Components of $\mathbf{x}$ can be continuous, categorical, curve, image, etc.

See for example the books [[Hastie et al., 2009](#), [Bishop, 2006](#)]. For R implementation, see for instance [[James et al., 2013](#)]

Mathematical context and terminology

- **Goal:** explain/predict an **output** $y \in \mathcal{Y}$ from **inputs** $\mathbf{x} \in \mathcal{X}$.
- **Available data:** $d_n = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ where the $y_i \in \mathcal{Y}$ denote the outputs and the $\mathbf{x}_i \in \mathcal{X}$ denote the inputs.

Terminology :

- ▷ y : the response, the output, the target.
- ▷ $\mathbf{x} = (x_1, \dots, x_p)$: the inputs, a vector of p predictors/features.
 - Use of bold print to emphasize that $\mathbf{x}$ is a vector.
- ▷ $x_1, \dots, x_j, \dots, x_p$: the components of vector $\mathbf{x}$, the p predictors.
 - Components of $\mathbf{x}$ can be continuous, categorical, curve, image, etc.

See for example the books [[Hastie et al., 2009](#), [Bishop, 2006](#)]. For R implementation, see for instance [[James et al., 2013](#)]

Two problems :

- ▷ If y is continuous, so y takes values in a real interval $\mathcal{Y} \subseteq \mathbb{R}$.
→ It is a **regression problem**.
- ▷ If y is categorical, so y takes values in a set of labels $\mathcal{Y} = \{1, \dots, K\}$.
→ It is a **classification problem**.

Statistical assumptions:

- One observation = one random variable $(\mathbf{X}, Y)$.
→ $d_n = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ represents n independent realizations of $(\mathbf{X}, Y)$.
- $Y \in \mathcal{Y}$: a random variable, the response, the target.
- $\mathbf{X} = (X_1, \dots, X_d) \in \mathcal{X}$: a random vector, the predictors, the features.
- $\mathbb{P}_{\mathbf{X}Y}$: the probability distribution of $(\mathbf{X}^T, Y)^T$.
- $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)\}$: a sample with n independent copies of $(\mathbf{X}, Y)$ where the inputs of the i th observation is $\mathbf{X}_i = (X_{i,1}, \dots, X_{i,p})$.

Convention: uppercase for random variables and lowercase for realisations, observed data.

Goal: **explain and predict** a variable Y based on the vector of variables $\mathbf{X}$.

⚠ Problem: $\mathbb{P}_{\mathbf{X}Y}$ is unknown.

Strategy in statistics:

Learn the relationship between $\mathbf{X}$ and Y **using a sample**

$$d_n = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\},$$

where $(\mathbf{x}_i, y_i)$ denotes a realisation of $(\mathbf{X}_i, Y_i)$.

Goal: **explain and predict** a variable Y based on the vector of variables $\mathbf{X}$.

⚠ Problem: $\mathbb{P}_{\mathbf{X}Y}$ is unknown.

Strategy in statistics:

Learn the relationship between $\mathbf{X}$ and Y **using a sample**

$$d_n = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\},$$

where $(\mathbf{x}_i, y_i)$ denotes a realisation of $(\mathbf{X}_i, Y_i)$.

Find a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ such as

$$f(\mathbf{x}_i) \approx y_i, \quad \text{for any } i = 1, \dots, n,$$

where

- $f(\mathbf{x}_i) = \hat{y}_i$ denotes the **prediction** for $\mathbf{x}_i$ in d_n ,
- y_i the **observed value** (also called the true value).

→ We want a *good* function = function that performs well with respect to a given performance criterion.

→ We have to choose a performance criterion to measure the loss (the error) between one prediction $f(\mathbf{x}) = \hat{y}$ and one observation y .

Find a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ such as

$$f(\mathbf{x}_i) \approx y_i, \quad \text{for any } i = 1, \dots, n,$$

where

- $f(\mathbf{x}_i) = \hat{y}_i$ denotes the **prediction** for $\mathbf{x}_i$ in d_n ,
- y_i the **observed value** (also called the true value).

→ We want a **good** function = function that **performs well with respect to a given performance criterion**.

→ We have to choose a **performance criterion** to measure the loss (the error) between one prediction $f(\mathbf{x}) = \hat{y}$ and one observation y .

The performance of $f : \mathcal{X} \rightarrow \mathcal{Y}$ is measured by using its **theoretical risk** $\mathcal{R}(f)$ defined as

$$\mathcal{R}(f) = \mathbf{E}_{\mathbb{P}_{\mathbf{X}\mathbf{Y}}}[\ell(Y, f(\mathbf{X}))]$$

where $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+$ is a measurable function satisfying

$$\begin{cases} \ell(y, \hat{y}) = 0 & \text{if } \hat{y} = y \\ \ell(y, \hat{y}) > 0 & \text{if } \hat{y} \neq y \end{cases}$$

The function ℓ is called **loss function**.

Remarks:

- The **loss** $\ell(\hat{y}, y)$ measures the discrepancy between the model prediction $\hat{y}$ and the observation y .
- Many loss functions exist. The most standard ones are probably
 - $\mathcal{L}^p$ losses for regression:

$$\ell(y, y') = |y - y'|^p, \quad p \in \mathbb{N}^*.$$

- 0-1 loss for classification:

$$\ell(y, y') = \mathbb{1}_{y \neq y'}.$$

- The **theoretical risk** $\mathcal{R}(f)$ is defined as the expected loss of the prediction function f under the underlying distribution $\mathbb{P}_{\mathbf{X}\mathbf{Y}}$. It therefore measures the *average loss of the prediction function f over the population*.
- The **theoretical risk** is also called the **generalization error**.

For a given loss function ℓ , we want the predictor with the lowest risk. That is, we seek a function f^* that minimizes the risk:

$$f^* = \operatorname{argmin}_{f \in \mathcal{F}} \mathcal{R}(f), \quad (1)$$

where $\mathcal{F}$ is a given model family. The predictor f^* is called the **optimal function** for the cost function ℓ in the family $\mathcal{F}$.

→ The minimum risk is attained. More precisely, there exists a $f^* \in \mathcal{F}$ that attains the minimum.

In regression, under the squared loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, the optimal predictor is the **regression function**:

$$f^*(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]. \quad (2)$$

In classification, under the 0–1 loss, the optimal predictor is the **Bayes classifier**:

$$f^*(\mathbf{x}) = \operatorname{argmax}_{k \in \{1, \dots, K\}} \mathbb{P}(Y = k \mid \mathbf{X} = \mathbf{x}). \quad (3)$$

BUT the optimal function f^* depends on the unknown probability distribution $\mathbb{P}_{\mathbf{X}\mathbf{Y}}$.

The goal is to construct a good estimate, or proxy, denoted by $f_n(\cdot, \mathcal{D}_n)$, of f^* such that

$$\mathcal{R}(f_n(\cdot, \mathcal{D}_n)) \approx \mathcal{R}(f^*).$$

Remarks:

- Statisticians use the observed data $\mathcal{D}_n$ to construct the estimate $f_n(\cdot, \mathcal{D}_n)$.
- For simplicity, we write $f_n = f_n(\cdot, \mathcal{D}_n)$.

→ A natural criterion for a learning procedure to be considered “good” is that its theoretical risk converges to the optimal risk as the number of observations tends to infinity. This property is called **consistency**.

BUT the optimal function f^* depends on the unknown probability distribution $\mathbb{P}_{\mathbf{X}\mathbf{Y}}$.

The goal is to construct a good estimate, or proxy, denoted by $f_n(\cdot, \mathcal{D}_n)$, of f^* such that

$$\mathcal{R}(f_n(\cdot, \mathcal{D}_n)) \approx \mathcal{R}(f^*).$$

Remarks:

- Statisticians use the observed data $\mathcal{D}_n$ to construct the estimate $f_n(\cdot, \mathcal{D}_n)$.
- For simplicity, we write $f_n = f_n(\cdot, \mathcal{D}_n)$.

→ A natural criterion for a learning procedure to be considered “good” is that its theoretical risk converges to the optimal risk as the number of observations tends to infinity. This property is called **consistency**.

We say that $f_n(., \mathcal{D}_n)$ is

- **weakly consistent** if for a given probability distribution $\mathbb{P}_{\mathbf{X}\mathbf{Y}}$, the theoretical risk converges in probability to the optimal error as the sample size grows, that is

$$\mathcal{R}(f_n(., \mathcal{D}_n)) \xrightarrow[n \rightarrow \infty]{\mathbb{P}} \mathcal{R}(f^*).$$

This means that, for any $\varepsilon > 0$,

$$\mathbb{P}(|\mathcal{R}(f_n(., \mathcal{D}_n)) - \mathcal{R}(f^*)| > \varepsilon) \xrightarrow[n \rightarrow \infty]{} 0.$$

Practical meaning: As the sample size increases, the probability that the estimator's risk differs substantially from the optimal risk approaches zero.

We say that $f_n(., \mathcal{D}_n)$ is

- **strongly consistent** if for a given probability distribution $\mathbb{P}_{\mathbf{X}\mathbf{Y}}$, the theoretical risk converges almost surely to the optimal error as the sample size grows, that is

$$\mathcal{R}(f_n(., \mathcal{D}_n)) \xrightarrow[n \rightarrow \infty]{a.s.} \mathcal{R}(f^*).$$

This means that

$$\mathbb{P} \left(\lim_{n \rightarrow \infty} \mathcal{R}(f_n(., \mathcal{D}_n)) = \mathcal{R}(f^*) \right) = 1.$$

Practical meaning: With probability one, as the sample size increases, the estimator's risk converges to the optimal risk.

A stronger property is **universal consistency**, which requires the consistency property (weak and/or strongly) to hold for **every possible** distribution $\mathbb{P}_{\mathbf{x}^Y}$.

Practical meaning: Universal consistency is a desirable theoretical property in machine learning. It guarantees that the risk of the learning procedure converges to the optimal risk as the sample size tends to infinity, regardless of the complexity or nature of the underlying data distribution.

In practice, since $\mathbb{P}_{\mathbf{X}Y}$ is unknown, we cannot compute the theoretical risk. We therefore estimate it using its **empirical counterpart**, called the **empirical risk**.

Given a loss function ℓ and a n -sample $\mathcal{L}_n = \{\mathbf{X}_i, Y_i\}_{i=1, \dots, n}$, the **empirical risk** of f_n is defined by

$$\mathcal{R}_{emp}(f_n, \mathcal{L}_n) = \frac{1}{n} \sum_{i=1}^n \ell(Y_i, f_n(\mathbf{X}_i)).$$

A natural idea is to estimate the empirical risk, computed on the training sample $\mathcal{D}_n$:

$$\mathcal{R}_{emp}(f_n, \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n \ell(Y_i, f_n(\mathbf{X}_i)),$$

- $\mathcal{R}_{emp}(f_n, \mathcal{D}_n)$ is called the **training empirical risk** or **training error**.
- $\mathcal{R}_{emp}(f_n, \mathcal{D}_n)$ often underestimate the theoretical risk.
- To obtain a more reliable estimate of the theoretical risk, we need **independent data** that were not used to fit the model.

Important remarks:

In supervised learning problems focused on prediction, the objective is formulated in terms of a **loss function** and its corresponding **risk function**, which we seek to minimize. This implies that:

- The selected estimator is optimal only with respect to a given criterion or, equivalently, a specific loss function ℓ .
- Consequently, this estimator may be suboptimal when evaluated under a alternative loss function ℓ' .
- The choice of the loss (or cost) function is a fundamental modeling decision.

To measure the performance of a model f_n , we need to choose an appropriate performance metric.

- Many commonly used performance metrics are empirical risks associated with particular loss functions.
- However, not all commonly used performance metrics are directly defined as empirical risks.
- The choice of metric depends on both **what we want to measure** and the **type of problem** (classification or regression).

- **In regression:** MSE, MAE, RMSE and MAPE are empirical risks associated with specific loss functions, while R^2 is a derived goodness-of-fit measure based on the squared prediction errors.
- **In classification:** Accuracy is directly related to the empirical risk under the 0–1 loss, while precision, recall, and the F_1 -score are derived from the confusion matrix.

Commonly used metrics in regression

- Mean Square Error, MSE :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - f_n(\mathbf{x}_i))^2.$$

→ Useful in optimization problems (differentiable), sensitive to outliers, emphasizes large errors.

- Mean Absolute Error, MAE :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - f_n(\mathbf{x}_i)|.$$

→ Sensitive to outliers.

- R^2 or coefficient of determination :

$$R^2 = 1 - \sum_{i=1}^n \frac{(y_i - f_n(\mathbf{x}_i))^2}{(y_i - \bar{y})^2}.$$

→ Note that it ranges from $-\infty$ to 1. The higher the R^2 , the better the model fits the data.

- Root Mean Square Error, RMSE :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - f_n(\mathbf{x}_i))^2}.$$

→ Useful in optimization problems (differentiable), sensitive to outliers, emphasizes large errors.

- Mean Absolute Percentage Error, MAPE :

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - f_n(\mathbf{x}_i)}{y_i} \right|.$$

→ Scale-invariant, sensitive to small values, unusable if there exists i such that $y_i = 0$.

Commonly used metrics in classification

Confusion matrix for classification:

		Model prediction			
		$\hat{y} = 1$	$\hat{y} = 2$	$\dots$	$\hat{y} = K$
Observation	$y = 1$	n_{11}	n_{12}	$\dots$	n_{1K}
	$y = 2$	n_{21}	n_{22}	$\dots$	n_{2K}
	$\vdots$				
	$y = K$	n_{K1}	n_{K2}	$\dots$	n_{KK}

Table 1: Confusion matrix ($\mathcal{Y} = \{1, \dots, K\}$)

Reading note: n_{ij} is the number of individuals belonging to class i classified as j by the model.

→ Most metrics are then derived from the confusion matrix.

Main metrics for classification:

- Error rate or misclassification rate :

$$\frac{\text{number of misclassified individuals}}{\text{total of individuals}} = \frac{1}{n} \sum_{i=1}^K \sum_{\substack{j=1 \\ i \neq j}}^K n_{ij} = \sum_{\substack{i,j=1 \\ i \neq j}}^n \frac{n_{ij}}{n}.$$

- Correct classification rate (accuracy) :

$$1 - \text{error rate} = 1 - \sum_{\substack{i,j=1 \\ i \neq j}}^K \frac{n_{ij}}{n} = \frac{\sum_{i=1}^K n_{ii}}{n}.$$

- Global misclassification cost :

$$\frac{\text{number of individuals of class } i \text{ classified as } j}{\text{total of individuals}} = \sum_{\substack{i,j=1 \\ i \neq j}}^K \frac{c_{ij} n_{ij}}{n},$$

where $c_{ij} > 0$ (for $i \neq j$) denotes the cost of misclassifying an individual of class i as j by the model.

Main metrics for binary classification:

		Model prediction	
		$\hat{y} = 1$	$\hat{y} = 0$
Observation	$y = 1$	True Positives TP or n_{11}	False Negatives FN or n_{12}
	$y = 0$	False Positives FP or n_{21}	True Negatives TN or n_{22}

Table 2: Confusion matrix

- True Positive Rate (Sensitivity or Recall): $TPR = \frac{TP}{TP+FN}$
- True Negative Rate (Specificity): $TNR = \frac{n_{22}}{n_{22}+n_{21}} = \frac{TN}{TN+FP}$
- False Negative Rate: $FNR = \frac{n_{12}}{n_{12}+n_{11}} = \frac{FN}{FN+TP}$
- False Positive Rate: $FPR = \frac{n_{21}}{n_{21}+n_{22}} = \frac{FP}{FP+TN}$
- Precision (True Positives among predicted positives): $= \frac{TP}{TP+FP}$

If we have an additional dataset $\mathcal{L}$, independent of the training data $\mathcal{D}_n$, we can estimate the performance of f_n on $\mathcal{L}$.

Problem: sometimes, we cannot have extra data.

Alternative solutions:

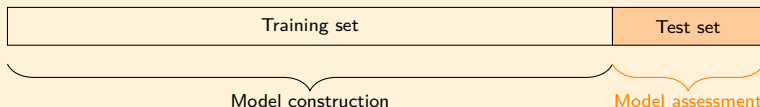
- use train/test (or hold-out),
- perform K -fold cross validation,
- use Monte Carlo approaches with train/test or K -fold cross validation.

Training/test or hold-out split

General idea: randomly split the dataset into a training set and a test set.

- Use the training sample $\mathcal{T}_{\mathcal{L}}$ only to train the model $f_n(\cdot, \mathcal{T}_{\mathcal{L}}) = f_n$;
- Use the test sample $\mathcal{T}_{\mathcal{A}}$ only to measure the model's performance

$$\mathcal{R}_{emp}(f_n, \mathcal{T}_{\mathcal{A}}) = \frac{1}{|\mathcal{T}_{\mathcal{A}}|} \sum_{i=1}^{|\mathcal{T}_{\mathcal{A}}|} \ell(Y_i, f_n(\mathbf{X}_i)),$$



Remarks:

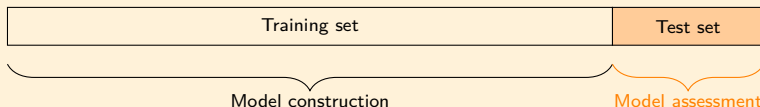
- Choose $n_{\text{train}} = |\mathcal{T}_{\mathcal{L}}|$ large enough to decrease the model's variance (i.e. obtain more stable estimates).
- Sometimes, $\mathcal{D}_n$ can be split into 3 distinct parts train/**validation**/test, if the model need to be calibrated (variable selection, hyperparameter tuning).

Training/test or hold-out split

General idea: randomly split the dataset into a training set and a test set.

- Use the training sample $\mathcal{T}_{\mathcal{L}}$ only to train the model $f_n(\cdot, \mathcal{T}_{\mathcal{L}}) = f_n$;
- Use the test sample $\mathcal{T}_{\mathcal{A}}$ only to measure the model's performance

$$\mathcal{R}_{emp}(f_n, \mathcal{T}_{\mathcal{A}}) = \frac{1}{|\mathcal{T}_{\mathcal{A}}|} \sum_{i=1}^{|\mathcal{T}_{\mathcal{A}}|} \ell(Y_i, f_n(\mathbf{X}_i)),$$



Remarks:

- Choose $n_{\text{train}} = |\mathcal{T}_{\mathcal{L}}|$ large enough to decrease the model's variance (i.e. obtain more stable estimates).
- Sometimes, $\mathcal{D}_n$ can be split into 3 distinct parts train/**validation**/test, if the model need to be calibrated (variable selection, hyperparameter tuning).

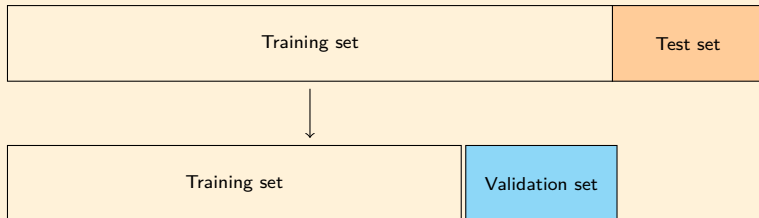
Steps:

1. Split the data into three distinct sets: train, validation, and test.
2. For each model: train on the training set, then evaluate performance on the validation set. → Select the model with the best performance.
3. Evaluate the performance of the final model on the test data.

Training set	Test set
--------------	----------

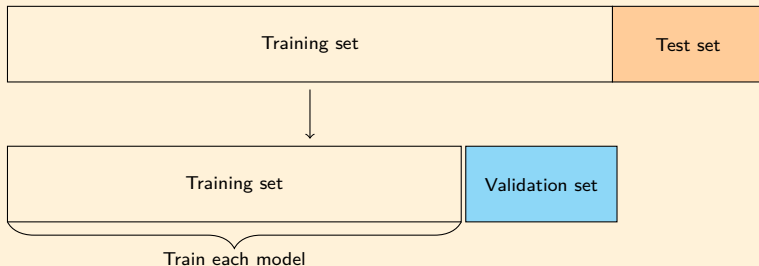
Steps:

1. Split the data into three distinct sets: train, validation, and test.
2. For each model: train on the training set, then evaluate performance on the validation set. → Select the model with the best performance.
3. Evaluate the performance of the final model on the test data.



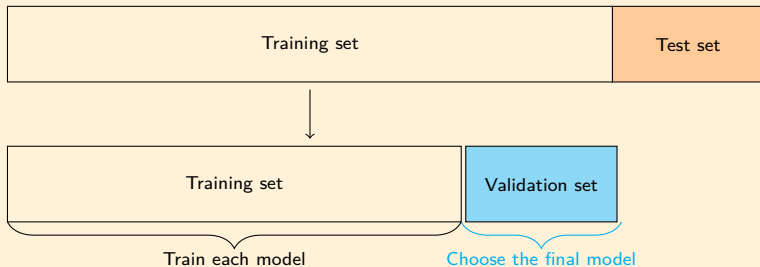
Steps:

1. Split the data into three distinct sets: train, validation, and test.
2. For each model: train on the training set, then evaluate performance on the validation set. → Select the model with the best performance.
3. Evaluate the performance of the final model on the test data.



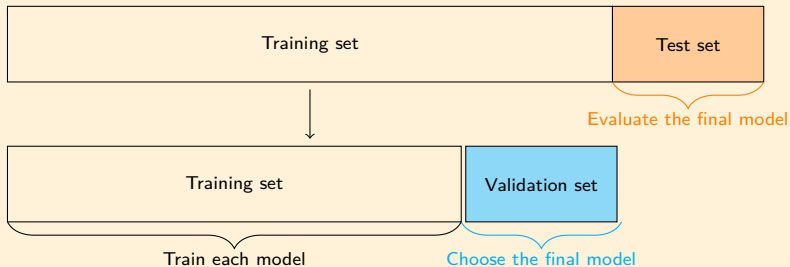
Steps:

1. Split the data into three distinct sets: train, validation, and test.
2. For each model: train on the training set, then evaluate performance on the validation set. → Select the model with the best performance.
3. Evaluate the performance of the final model on the test data.



Steps:

1. Split the data into three distinct sets: train, validation, and test.
2. For each model: train on the training set, then evaluate performance on the validation set. → Select the model with the best performance.
3. Evaluate the performance of the final model on the test data.



Algorithm 1: Train-Validation-Test Split

Input: Dataset $\mathcal{D}_n$, performance metric Err and proportions

$\alpha_{\text{train}}, \alpha_{\text{val}}, \alpha_{\text{test}}$ (with $\alpha_{\text{train}} + \alpha_{\text{val}} + \alpha_{\text{test}} = 1$).

1. Data splitting: Randomly split $\mathcal{D}_n$ into three disjoint subsets: $\mathcal{D}_{n\text{train}}$, $\mathcal{D}_{n\text{val}}$, and $\mathcal{D}_{n\text{test}}$, with proportions α_{train} , α_{val} , and α_{test} ;

2. Model training: Train the learning procedure on $\mathcal{D}_{n\text{train}}$ to obtain the predictor f_n ;

3. Model selection: Evaluate f_n on $\mathcal{D}_{n\text{val}}$: $\text{Err}_{\text{val}} = \text{Err}(f_n, \mathcal{D}_{n\text{val}})$;

Use Err_{val} to select or tune the model;

4. Final evaluation:

Once the model is selected, evaluate its performance on $\mathcal{D}_{n\text{test}}$:

$\text{Err}_{\text{test}} = \text{Err}(f_n, \mathcal{D}_{n\text{test}})$;

Output: Err_{test}

Remarks:

- In classification, when the distribution of Y is imbalanced, stratification on Y can be used.
- ✓ Simple and is easy to implement.
- ✗ The performance obtained depends on how the data are split and can therefore vary substantially from one split to another.
- ✗ Train/validation/test split is not very appropriate when the dataset $\mathcal{D}_n$ is small.

General idea:

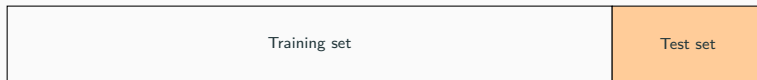
1. Split the data into a train set and a test set.
2. Then split the train set into k folds and perform training/validation on each fold.

Motivations:

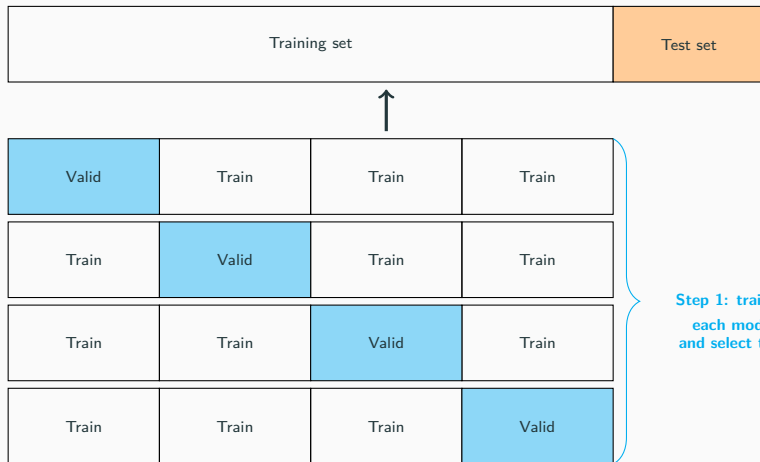
- Use as much data as possible to train the model.
- Reduce the impact of sampling when splitting into train/test sets.

Exemple pour $k = 4$

Randomly split the dataset into a training set and a test set. Then, randomly split the training set into k equal parts.

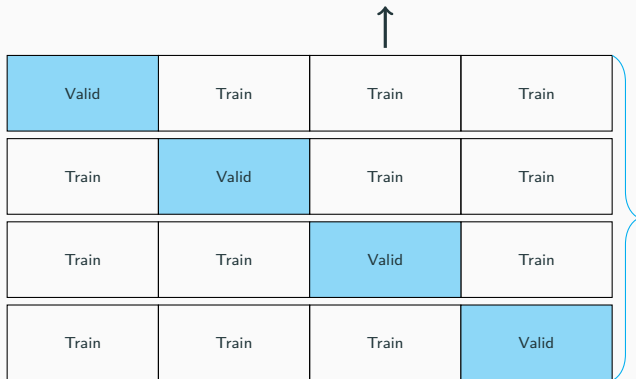
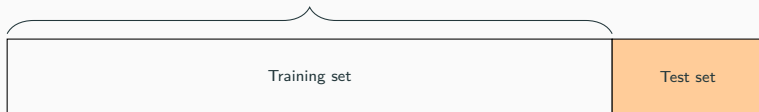


Exemple pour $k = 4$



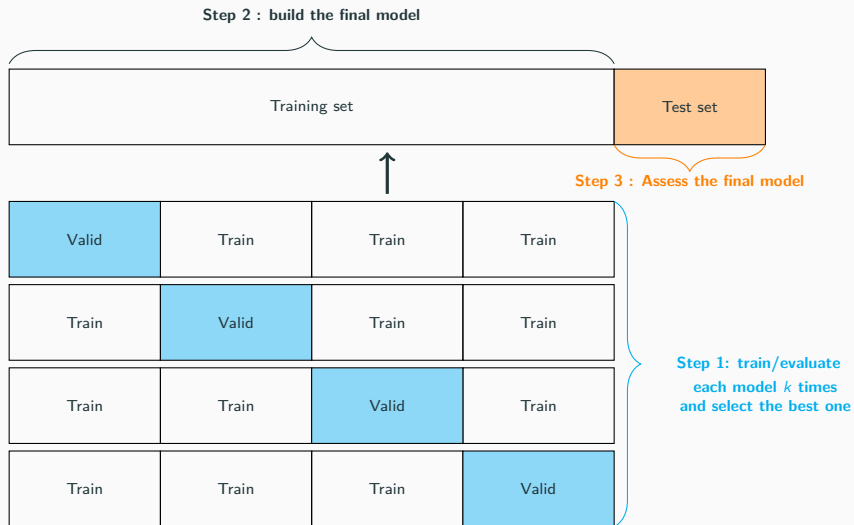
Exemple pour $k = 4$

Step 2 : build the final model



Step 1: train/evaluate
each model k times
and select the best one

Exemple pour $k = 4$



Algorithm 2: K -fold Cross-Validation

Input: Dataset $\mathcal{D}_n$, number of folds K , performance metric Err

Randomly partition $\mathcal{D}_n$ into K disjoint folds $\mathcal{D}_{n1}, \dots, \mathcal{D}_{nK}$;

for $k = 1, \dots, K$ **do**

 Train the learning procedure on $\mathcal{D}_n \setminus \mathcal{D}_{nk}$ to obtain $f_n^{(-k)}$;
 Compute $\text{Err}_k = \text{Err}(f_n^{(-k)}, \mathcal{D}_{nk})$;

Compute $\text{Err}_{CV} = \frac{1}{K} \sum_{k=1}^K \text{Err}_k$;

Output: Err_{CV}

- In practice, K is chosen by the user (often $K = 3, 5$, or 10).
- Leave-one-out cross-validation (LOOCV) = K -fold cross-validation with $K = n$. Principle: only one observation is left out of $\mathcal{D}_n$ at each iteration, resulting in n fitted models.
 - Very expensive to compute when n large.
- ✓ More adapted than train/validation/test when n is small.
- ✗ Could be time consuming.
- ✗ The performance obtained depends on how the data are split and can therefore vary substantially from one split to another.

- In the previous approaches, the choice of the final model may depend strongly on how the data are partitioned, especially when the dataset is small or, in classification, when a class is highly underrepresented.

Estimation by Monte Carlo:

Choose an approach (train/test, train/validation/test, or K -fold cross-validation), repeat it independently B times, and compute the average performance.



Bishop, C. M. (2006).

Pattern recognition.

Machine learning, 128(9).



Hastie, T., Tibshirani, R., and Friedman, J. (2009).

The elements of statistical learning: data mining, inference, and prediction.

Springer Science & Business Media.



James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013).

An introduction to statistical learning, volume 112.

Springer.